EXPERIMENT-70(HDOC)

C Programming

Program: Dynamic Matrices — Addition, Subtraction and Multiplication Using Pointer Notation

Step 1: Understand the Problem Statement

- Input: The dimensions (rows and columns, each between 1 and 10) and the elements of two matrices A and B.
- Output: Matrix A, matrix B and the results of A + B, A − B and A × B. If an operation is not possible for the given sizes, a message explains why.
- Dynamic memory: each matrix is a pointer to a pointer (`int **`). `allocateMatrix()` first allocates an array of row pointers with `calloc()` and then one block of integers for every row with `malloc()`. If any request fails, the memory already obtained is freed and NULL is returned.
- Pointer notation: element (i, j) is reached as `*(*(m + i) + j)` instead of `m[i][j]`, and the address of an element is `*(m + i) + j` (used with `scanf()`).
- Each operation is a separate function that allocates and returns its own result matrix: `addMatrices()`, `subtractMatrices()` and `multiplyMatrices()`. Supporting functions are `readMatrix()`, `displayMatrix()`, `allocateMatrix()` and `freeMatrix()`.
- Memory release: every matrix (A, B and each result) is released by `freeMatrix()`, which frees every row first and then the array of row pointers, so that nothing is leaked. The program was also run under AddressSanitizer (which includes the leak checker) and reported no memory errors or leaks.

Rule(s) used: Addition and subtraction need matrices of the same size: C[i][j] = A[i][j] ± B[i][j]. Multiplication needs the number of columns of A to equal the number of rows of B: for A (r1 × c1) and B (c1 × c2), C[i][j] = Σ A[i][k] × B[k][j] for k = 0 to c1−1, and C is r1 × c2.

Step 2: Test Case Table

Test Case	Input(s)	Expected Output(s)	Actual Output(s)	Result
1	`A (2x2): [[1, 2], [3, 4]]` `B (2x2): [[5, 6], [7, 8]]`	A + B = [[6, 8], [10, 12]] A - B = [[-4, -4], [-4, -4]] A x B = [[19, 22], [43, 50]]	A + B = [[6, 8], [10, 12]] A - B = [[-4, -4], [-4, -4]] A x B = [[19, 22], [43, 50]]	Pass
2	`A (2x3): [[1, 2, 3], [4, 5, 6]]` `B (3x2): [[7, 8], [9, 10], [11, 12]]`	Addition and subtraction not possible: matrices must have the same dimensions. A x B = [[58, 64], [139, 154]]	Addition and subtraction not possible: matrices must have the same dimensions. A x B = [[58, 64], [139, 154]]	Pass
3	`A (2x3): [[1, 2, 3], [4, 5, 6]]` `B (2x3): [[6, 5, 4], [3, 2, 1]]`	A + B = [[7, 7, 7], [7, 7, 7]] A - B = [[-5, -3, -1], [1, 3, 5]] Multiplication not possible: columns of A must equal rows of B.	A + B = [[7, 7, 7], [7, 7, 7]] A - B = [[-5, -3, -1], [1, 3, 5]] Multiplication not possible: columns of A must equal rows of B.	Pass

Test Case	Input(s)	Expected Output(s)	Actual Output(s)	Result
4	`A (1x1): [[3]]` `B (1x1): [[4]]`	A + B = [[7]] A - B = [[-1]] A x B = [[12]]	A + B = [[7]] A - B = [[-1]] A x B = [[12]]	**Pass**
5	`A (3x3): [[1, 2, 3], [4, 5, 6], [7, 8, 9]]` `B (3x3): [[1, 0, 0], [0, 1, 0], [0, 0, 1]]`	A + B = [[2, 2, 3], [4, 6, 6], [7, 8, 10]] A - B = [[0, 2, 3], [4, 4, 6], [7, 8, 8]] A x B = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]	A + B = [[2, 2, 3], [4, 6, 6], [7, 8, 10]] A - B = [[0, 2, 3], [4, 4, 6], [7, 8, 8]] A x B = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]	**Pass**
6	`Dimensions of A: 0 2`	Invalid dimensions. Enter values between 1 and 10.	Invalid dimensions. Enter values between 1 and 10.	**Pass**

Step 3: Algorithm

1. Start the program.

2. Read the dimensions of A (r1 × c1) and of B (r2 × c2); if any value is outside 1 to 10, display an error message and stop.

3. Allocate A and B with `allocateMatrix()`; if either allocation fails, free what was obtained, report the failure and stop.

4. Read the elements of A and B row by row with `readMatrix()` and display both matrices.

5. If r1 = r2 and c1 = c2: call `addMatrices()`, display the result and free it; then call `subtractMatrices()`, display the result and free it. Otherwise display that addition and subtraction are not possible.

6. If c1 = r2: call `multiplyMatrices()` (three nested loops, each element `*(*(prod + i) + j) += *(*(a + i) + k) * *(*(b + k) + j)`), display the result and free it. Otherwise display that multiplication is not possible.

7. Free matrices A and B with `freeMatrix()` and display that all memory has been released.

8. Stop the program.

Evaluation against Step 2 test cases: for the 2 × 2 matrices [[1, 2], [3, 4]] and [[5, 6], [7, 8]] the sum is [[6, 8], [10, 12]], the difference is [[−4, −4], [−4, −4]] and the product is [[19, 22], [43, 50]]. A 2 × 3 matrix and a 3 × 2 matrix cannot be added or subtracted but can be multiplied, giving the 2 × 2 matrix [[58, 64], [139, 154]]. Two 2 × 3 matrices can be added and subtracted but not multiplied, because 3 columns do not match 2 rows. The 1 × 1 case gives 7, −1 and 12, multiplying a 3 × 3 matrix by the identity matrix returns it unchanged, and a dimension of 0 is rejected — identical to the Expected Output(s) column, confirming the algorithm is correct before coding.

Step 4: C Program

```
/* HDOC Day-71: Dynamic allocation of two matrices; addition, subtraction and
   multiplication using POINTER NOTATION.

   A matrix is a pointer to an array of row pointers (int **). Memory comes
```

```
   from malloc()/calloc() at run time, element (i, j) is reached as
   *(*(m + i) + j) instead of m[i][j], every operation is a separate function
   that returns a newly allocated result matrix, and everything that was
   allocated is released with free() before the program ends.
*/

#include <stdio.h>
#include <stdlib.h>

#define MAX_DIM 10

/* Function prototypes */
int  **allocateMatrix(int rows, int cols);
void   freeMatrix(int **m, int rows);
void   readMatrix(int **m, int rows, int cols, char name);
void   displayMatrix(int **m, int rows, int cols, const char *title);
int  **addMatrices(int **a, int **b, int rows, int cols);
int  **subtractMatrices(int **a, int **b, int rows, int cols);
int  **multiplyMatrices(int **a, int **b, int r1, int c1, int c2);
int    readDimensions(int *rows, int *cols, char name);

int main() {
    int r1, c1, r2, c2;
    int **A, **B, **result;

    if (!readDimensions(&r1, &c1, 'A') || !readDimensions(&r2, &c2, 'B')) {
        printf("Invalid dimensions. Enter values between 1 and %d.\n", MAX_DIM);
        return 1;
    }

    A = allocateMatrix(r1, c1);
    B = allocateMatrix(r2, c2);
    if (A == NULL || B == NULL) {
        printf("Memory allocation failed.\n");
        if (A != NULL) freeMatrix(A, r1);
        if (B != NULL) freeMatrix(B, r2);
        return 1;
    }

    readMatrix(A, r1, c1, 'A');
    readMatrix(B, r2, c2, 'B');
    printf("\n");
    displayMatrix(A, r1, c1, "Matrix A:");
    displayMatrix(B, r2, c2, "Matrix B:");

    /* addition and subtraction need matrices of the same size */
    if (r1 == r2 && c1 == c2) {
        result = addMatrices(A, B, r1, c1);
        if (result != NULL) {
            displayMatrix(result, r1, c1, "Addition (A + B):");
            freeMatrix(result, r1);
        }
        result = subtractMatrices(A, B, r1, c1);
        if (result != NULL) {
            displayMatrix(result, r1, c1, "Subtraction (A - B):");
            freeMatrix(result, r1);
        }
    } else {
        printf("Addition and subtraction not possible: matrices must have the same dimensions.\n\n");
    }

    /* multiplication needs columns of A = rows of B */
    if (c1 == r2) {
        result = multiplyMatrices(A, B, r1, c1, c2);
```

```
        if (result != NULL) {
            displayMatrix(result, r1, c2, "Multiplication (A x B):");
            freeMatrix(result, r1);
        }
    } else {
        printf("Multiplication not possible: columns of A must equal rows of B.\n\n");
    }

    freeMatrix(A, r1);
    freeMatrix(B, r2);
    printf("All dynamically allocated memory has been released.\n");

    return 0;
}

/* Reads and validates the dimensions of one matrix */
int readDimensions(int *rows, int *cols, char name) {
    printf("Enter rows and columns of matrix %c: ", name);
    if (scanf("%d %d", rows, cols) != 2)
        return 0;
    return *rows >= 1 && *rows <= MAX_DIM && *cols >= 1 && *cols <= MAX_DIM;
}

/* Allocates a rows x cols matrix: one block of row pointers, then one block
   of integers for every row. Returns NULL (and frees what was already
   allocated) if any request fails. */
int **allocateMatrix(int rows, int cols) {
    int i, j;
    int **m = (int **)calloc(rows, sizeof(int *));
    if (m == NULL)
        return NULL;

    for (i = 0; i < rows; i++) {
        *(m + i) = (int *)malloc(cols * sizeof(int));
        if (*(m + i) == NULL) {
            for (j = 0; j < i; j++)
                free(*(m + j));
            free(m);
            return NULL;
        }
    }
    return m;
}

/* Releases every row and then the array of row pointers */
void freeMatrix(int **m, int rows) {
    int i;
    for (i = 0; i < rows; i++)
        free(*(m + i));
    free(m);
}

/* Reads the matrix row by row using pointer notation */
void readMatrix(int **m, int rows, int cols, char name) {
    int i, j;
    printf("Enter matrix %c (%d x %d), one row per line:\n", name, rows, cols);
    for (i = 0; i < rows; i++) {
        printf("Row %d: ", i + 1);
        for (j = 0; j < cols; j++)
            scanf("%d", *(m + i) + j);       /* address of element (i, j) */
    }
}

/* Displays the matrix using pointer notation */
```

```
void displayMatrix(int **m, int rows, int cols, const char *title) {
    int i, j;
    printf("%s\n", title);
    for (i = 0; i < rows; i++) {
        for (j = 0; j < cols; j++)
            printf("%4d ", *(*(m + i) + j));
        printf("\n");
    }
    printf("\n");
}

/* Returns a new matrix holding a + b */
int **addMatrices(int **a, int **b, int rows, int cols) {
    int i, j;
    int **sum = allocateMatrix(rows, cols);
    if (sum == NULL)
        return NULL;
    for (i = 0; i < rows; i++)
        for (j = 0; j < cols; j++)
            *(*(sum + i) + j) = *(*(a + i) + j) + *(*(b + i) + j);
    return sum;
}

/* Returns a new matrix holding a - b */
int **subtractMatrices(int **a, int **b, int rows, int cols) {
    int i, j;
    int **diff = allocateMatrix(rows, cols);
    if (diff == NULL)
        return NULL;
    for (i = 0; i < rows; i++)
        for (j = 0; j < cols; j++)
            *(*(diff + i) + j) = *(*(a + i) + j) - *(*(b + i) + j);
    return diff;
}

/* Returns a new r1 x c2 matrix holding a x b (a is r1 x c1, b is c1 x c2) */
int **multiplyMatrices(int **a, int **b, int r1, int c1, int c2) {
    int i, j, k;
    int **prod = allocateMatrix(r1, c2);
    if (prod == NULL)
        return NULL;
    for (i = 0; i < r1; i++) {
        for (j = 0; j < c2; j++) {
            *(*(prod + i) + j) = 0;
            for (k = 0; k < c1; k++)
                *(*(prod + i) + j) += *(*(a + i) + k) * *(*(b + k) + j);
        }
    }
    return prod;
}
```

Step 5: Compile, Execute and Evaluate

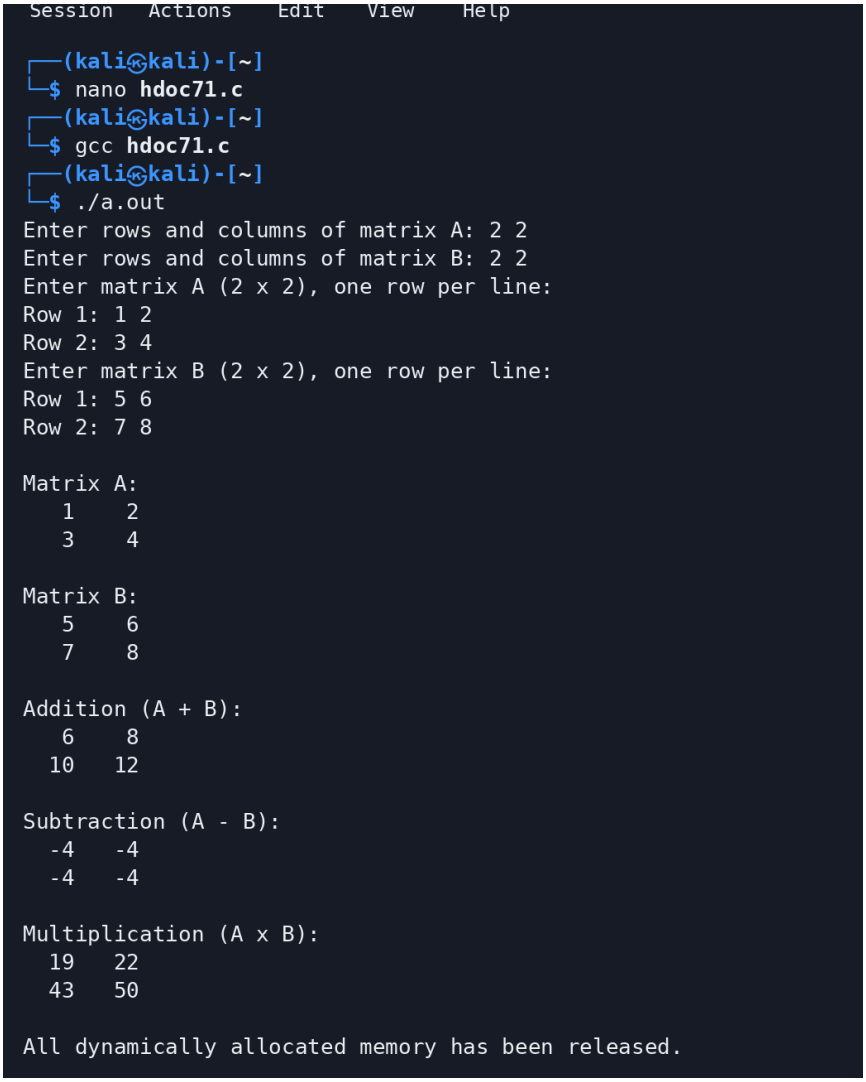
The program was compiled using the GCC/cc compiler and executed from the Linux terminal using the following commands:

cc hdoc71.c	(compile)
./a.out	(execute)

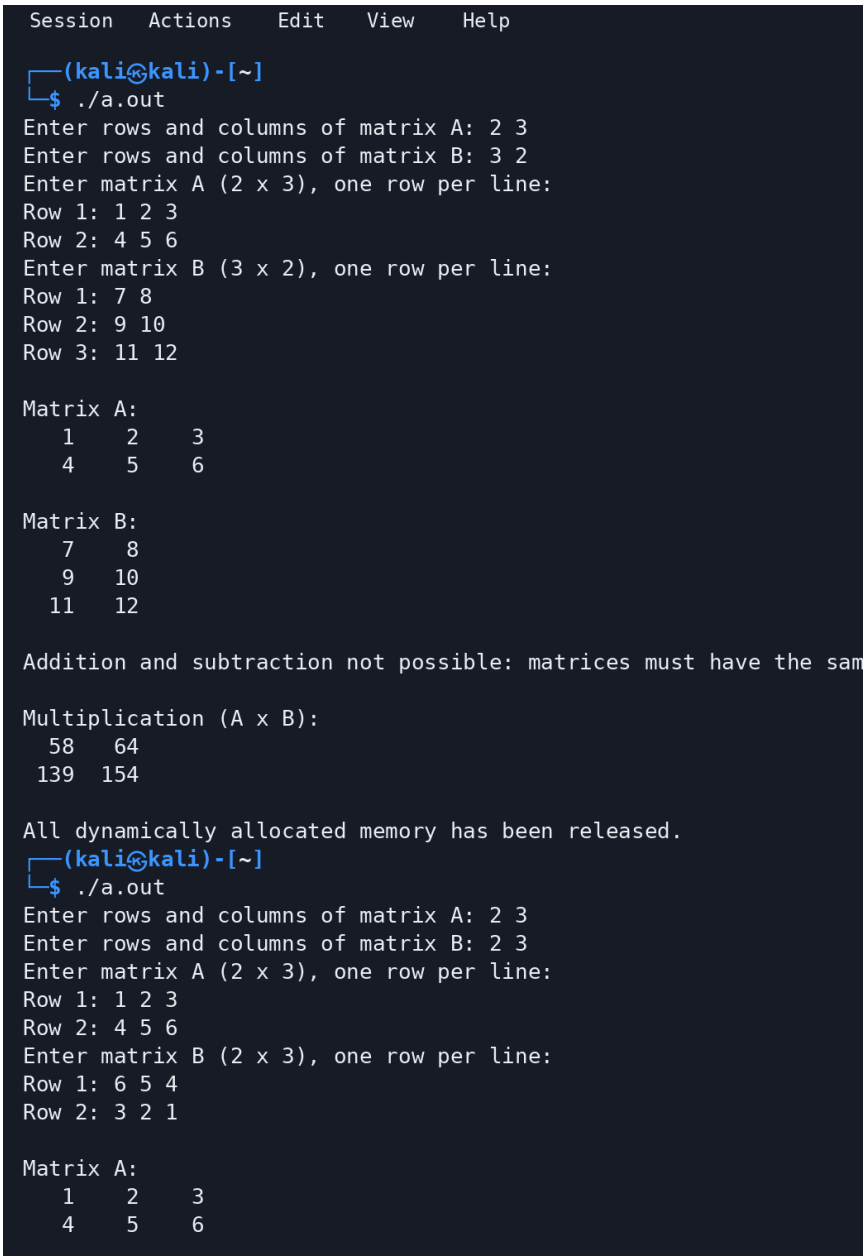
All 6 test cases prepared in Step 2 were executed against the compiled program. The Actual Output(s) obtained matched the Expected Output(s) in every case, so all test cases have been marked as Pass (see table in Step 2).

Step 6: Compilation and Execution Screenshot

Screenshot 1 of 4



Screenshot 2 of 4



Screenshot 3 of 4

```
 Session   Actions    Edit    View    Help

Matrix B:
   6    5    4
   3    2    1

Addition (A + B):
   7    7    7
   7    7    7

Subtraction (A - B):
  -5   -3   -1
   1    3    5

Multiplication not possible: columns of A must equal rows of B.

All dynamically allocated memory has been released.
┌──(kali㉿kali)-[~]
└─$ ./a.out
Enter rows and columns of matrix A: 1 1
Enter rows and columns of matrix B: 1 1
Enter matrix A (1 x 1), one row per line:
Row 1: 3
Enter matrix B (1 x 1), one row per line:
Row 1: 4

Matrix A:
   3

Matrix B:
   4

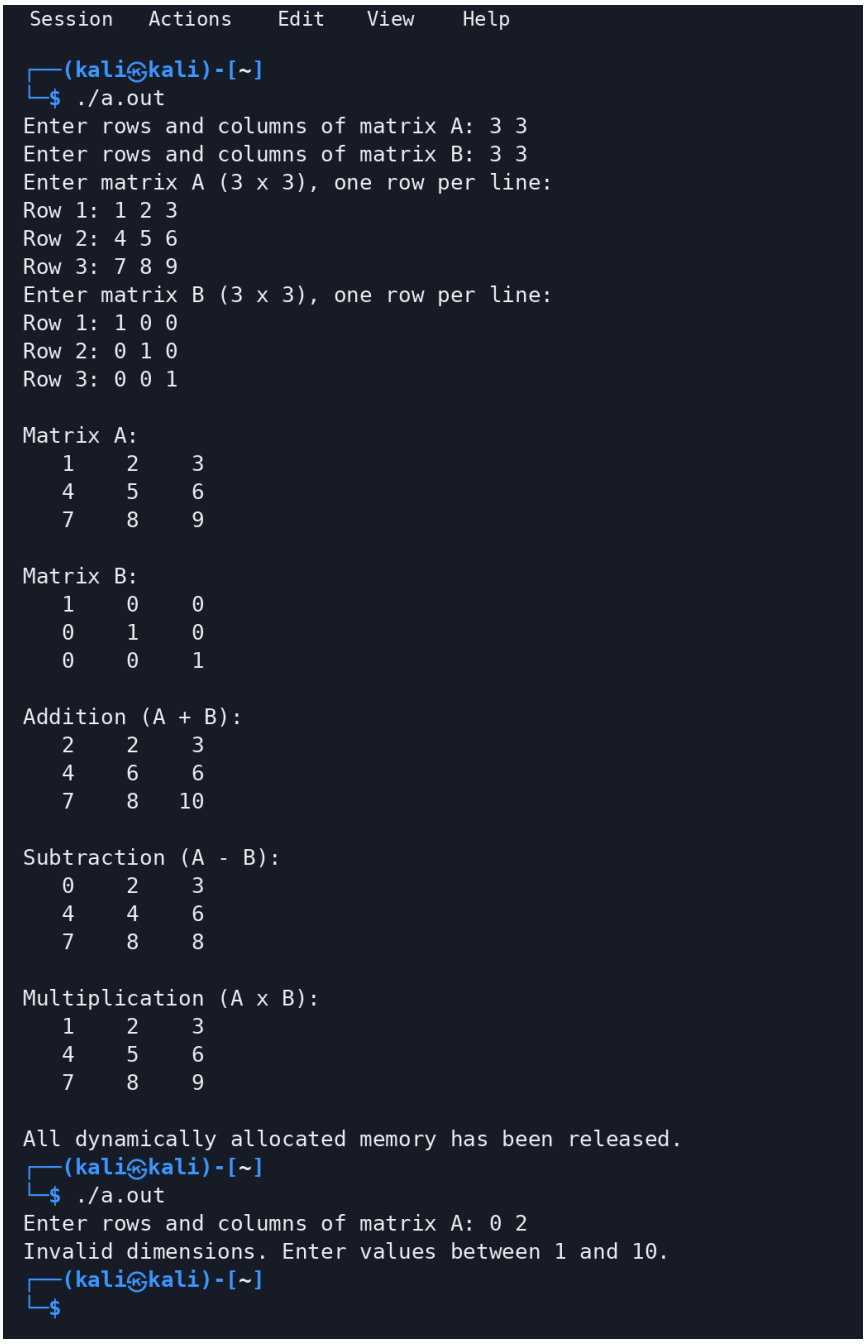
Addition (A + B):
   7

Subtraction (A - B):
  -1

Multiplication (A x B):
  12

All dynamically allocated memory has been released.
```

Screenshot 4 of 4



Conclusion

In this experiment, two matrices were allocated dynamically as arrays of row pointers using malloc() and calloc(), filled and displayed with pointer notation, and combined by three separate functions that perform addition, subtraction and multiplication and each return a newly allocated result matrix, with the size rules for each operation checked before it is attempted. All memory (both input matrices and every result) was released with free(); the program was compiled with gcc, tested on six cases including incompatible sizes and invalid input with every result matching an independent calculation, and an AddressSanitizer run showed no memory errors or leaks.